به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴) ۵ اردیبهشت ۹۸

دانشگاه صنعتی شریف مدرس: مهدی صفرنژاد

---

# آزمون میان‌ترم

زمان این امتحان ۱۵۰ دقیقه است. سؤالات امتحان در چهار صفحه (دو برگهٔ پشت و رو) قرار دارند. مجموع نمرات سؤالات ۱۲۰ نمره است که ~~۱۰۰~~ ۳۰ نمرهٔ آن امتیازی است. توضیحات ابتدای سؤالات را به دقت بخوانید و پاسخ هر سؤال را در برگه‌های جداگانه بنویسید. پاسخ بخش‌های مختلف هر سؤال را نیز به ترتیب در برگهٔ پاسخ‌نامه بنویسید. امیدوارم تا این‌جای کلاس با تفکر الگوریتمی آشنا شده باشید و با همین تفکر به سؤالات امتحان به خوبی پاسخ دهید.

---

**سؤال ۱. [۳۳ نمره] سؤال‌های جواب کوتاه.** در سؤال‌های این قسمت معمولاً یک جواب کوتاه (۱ نمره) و یک توضیح کوتاه حدوداً بین یک تا دو خط (۲ نمره) مدنظر است؛ مگر این که صورت آن قسمت به صراحت از شما خواستهٔ دیگری داشته باشد.

آ. **[۳ نمره]** آیا مرتبهٔ زمانی یک الگوریتم برابر با تعداد مراحل آن الگوریتم در مدل محاسباتی‌اش است؟

**پاسخ:** خیر، زیرا ممکن است زمان اجرای یک مرحله در مدل محاسبات $O(1)$ نباشد (مثل ماشین پایتون که در کلاس گفته شد). این سؤال در کوییز ۱ نیز مطرح شده بود.

ب. **[۳ نمره]** آیا الگوریتمی وجود دارد که $k$ لیست مرتب، که هر لیست به طول $n$ است، را در زمان $O(nk)$ (در یک لیست مرتب به طول $nk$) ادغام کند؟

**پاسخ:** خیر، در حالت حدی فرض کنید که لیست‌ها تک عنصره باشند ($n = ۱$). در این حالت ادغام $k$ لیست تک‌عنصره، همانند مرتب‌سازی $k$ عدد می‌شود که نمی‌توان (در مدل مقایسه) مرتب‌سازی را در $O(k)$ انجام داد.

پ. **[۳ نمره]** آرایهٔ $A$ یک هرم بیشینه را نشان می‌دهد که یک عنصر آن تغییر کرده است و رأس مربوط به آن دیگر خاصیت هرم بیشینهٔ رأسی را ندارد. ساختار درختی این هرم را ترسیم کنید و رأس گفته شده را در آن مشخص کنید.

$$A = [۱۸, ۱۵, ۱۲, ۹, ۱۴, ۱۳, ۸, ۷, ۶]$$

**پاسخ:** عنصر ۱۲ خاصیت هرم بیشینه را ندارد (زیرا ۱۳ فرزند آن است). توجه کنید ۱۳ خاصیت هرم بیشینهٔ رأسی را دارد زیرا از فرزندان خود بزرگتر است (زیرا فرزندی ندارد).

**نمره‌دهی:** اگر رأس ۱۳ را به جای ۱۲ نوشته باشند، ۱٫۵ نمره تعلق می‌گیرد.

ت. **[۳ نمره]** آیا می‌توان در زمان خطی یک هرم بیشینه را به یک هرم کمینه تبدیل کرد؟

**پاسخ:** بله، هر آرایهٔ دلخواه از اعداد را می‌توان در زمان خطی به هرم کمینه تبدیل کرد.

ث. **[۳ نمره]** آیا مرتب‌سازی هرمی روی یک آرایهٔ مرتب در زمان $O(n)$ انجام می‌شود؟

**پاسخ:** خیر، زیرا در مرتب‌سازی هرمی در $n$ مرحله عنصر بیشینه استخراج می‌شود که هر کدام از این استخراج‌ها $\Theta(\log n)$ زمان می‌گیرد.

ج. **[۳ نمره]** در یک درخت دودویی جست‌وجو، تعدادی عنصر با کلید یکسان $a$ درج شده‌اند. آیا ممکن است یک رأس با کلید $a$ در این درخت وجود داشته باشد که کلیدهای والد و فرزندان آن $a$ نباشند؟ (اگر جواب خیر است دلیل آن را توضیح دهید و اگر جواب بله است یک مثال بزنید)

**پاسخ:** بله. یک مثال سه رأسی برای آن وجود دارد که کلید ریشه ۲، فرزند چپ ریشه دارای کلید ۱ باشد و خود این رأس فرزند راستی داشته باشد که دارای کلید ۲ باشد.

چ. **[۳ نمره]** آیا یک درخت دودویی جست‌وجوی غیرمتوازن را می‌توان به یک درخت AVL با حداکثر $O(\log n)$ دوران تبدیل کرد؟

**پاسخ:** خیر، زیرا هر دوران ارتفاع را حداکثر یک واحد کم می‌کند و مثال‌هایی از درخت‌های دودویی جست‌وجوی غیرمتوازن وجود دارد که ارتفاع آن‌ها $\Theta(n)$ است.

**نمره‌دهی:**

- کسانی که به $\mathcal{O}(n)$ بودن اشاره کرده‌اند یک نمره گرفته‌اند.
- اشاره به اینکه با هر دوران می‌توان ارتفاع را حداکثر یک واحد کاهش داد یک نمره دارد و کسانی که به این موضوع اشاره نکرده‌اند یک نمره از دست داده‌اند.

ح. **[۳ نمره]** آیا یک الگوریتم مونت‌کارلو همیشه جواب درست را برمی‌گرداند؟

**پاسخ:** خیر، الگوریتم‌های مونت‌کارلو الگوریتم‌های احتمالاً درست هستند که درستی خروجی الگوریتم احتمالی است ولی همیشه سریع هستند. در کوییز سوم مشابه این سؤال وجود دارد.

خ. **[۳ نمره]** یک درخت قرمز_سیاه با $n^{۱۰}$ رأس داریم. یک جست‌وجو در این درخت (به صورت مجانبی) چه زمانی دارد؟

**پاسخ:** ارتفاع درخت (به خاطر توازن) $O(\log n) = O(۱۰ \cdot \log n) = O(\log(n^{۱۰}))$ است و جست‌وجو در همین مرتبه انجام می‌شود.

**نمره‌دهی:**

- کسانی که به $\mathcal{O}(\log n)$ بودن جست‌وجو در درخت قرمز_سیاه اشاره کرده‌اند یک نمره گرفته‌اند.

- کسانی که $n^{۱۰}$ را در رابطه بالا جایگذاری نکرده‌اند، یا جواب را به صورت ساده نشده $\mathcal{O}(۱۰ \log n)$ ارائه داده‌اند یک نمره از دست داده‌اند.

د. **[۳ نمره]** آیا ۷ عنصر را می‌توان با ۱۲ مقایسه مرتب کرد؟

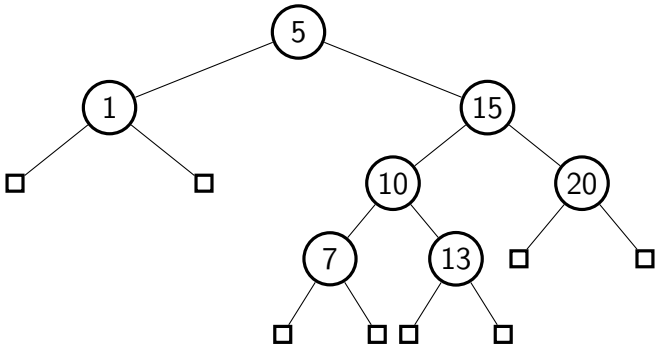
**پاسخ:** خیر، زیرا بنابر استدلال درخت تصمیم به $\lceil \log_۲(۵۰۴۰) \rceil = \lceil \log_۲(۷!) \rceil$ مقایسه نیاز داریم.

**نمره‌دهی:**

- کسانی که به درخت تصمیم اشاره کرده‌اند یک نمره دریافت کرده‌اند.
- کسانی که در محاسبه اشتباه کرده‌اند یک نمره از دست داده‌اند.
- کسانی که گفته‌اند مرتب‌سازی در بهترین حالت در $\mathcal{O}(n \log n)$ است و محاسبه را نیز با $n \log n$ انجام داده‌اند یک نمره از دست داده‌اند.

ذ. **[۳ نمره]** رأس‌های درخت زیر را با رنگ‌های قرمز و سیاه طوری رنگ کنید که خاصیت‌های درخت قرمز–سیاه را داشته باشد. رنگ برگ‌ها را نیز مشخص کنید.

**پاسخ:** رنگ رأس‌های ۱۵، ۷ و ۱۳ قرمز و رنگ بقیه سیاه است. این سؤال در کوییز دوم بوده است.




**نمره‌دهی:**

- در این قسمت برای رنگ برگ‌ها یک نمره در نظر گرفته شده است.

**سؤال ۲.** **[۲۰ نمره]** **تحلیل مجانبی.** در هر قسمت علاوه بر پاسخ درست، روابطی که استفاده کردید را نوشته و به طور مختصر توضیح دهید.

آ. **[۵ نمره]** توابع زیر را از لحاظ مجانبی مرتب کنید.

$$f_۱(n) = (۲n)!, f_۲(n) = \binom{۲n}{n}, f_۳(n) = ۲^{۲n}, f_۴(n) = (۲n)^n, f_۵(n) = n^{۲n}$$

**پاسخ:** ترتیب توابع از کوچک به بزرگ و از چپ به راست:

$$(f_۲, f_۳, f_۴, f_۱, f_۵)$$

این ترتیب با استفاده از تقریب استرلینگ به راحتی به دست می‌آید. به طور مثال باید $f_۲$ را به صورت $\frac{(۲n)!}{n!\cdot n!}$ نوشت و سپس هر کدام از عبارت‌های فاکتوریل را با تقریب استرلینگ ساده کرد. با این کار می‌توان نشان داد $f_۲$ از بقیه توابع (به صورت مجانبی) کوچک‌تر است. همچنین $f_۱$ بین $f_۴$ و $f_۵$ قرار دارد. مشابه این سؤال در تمرین اول داده شده است.

**نمره‌دهی:** اگر ترتیب درست باشد کل نمره داده می‌شود. اگر ترتیب ۲ تابع اشتباه باشد یک نمره کسر می‌شود. برای ترتیب اشتباه ۳ تابع، ۳ نمره و برای ترتیب اشتباه ۴ تابع ۴ نمره کم می‌شود. ترتیب اشتباه ۵ تابع هم نمره‌ای نمی‌گیرد.

ب. **[۵ نمره]** رابطهٔ بازگشتی زیر را حل کنید. (راهنمایی: از تغییر متغیر $n = ۲^m$ استفاده کنید)

$$T(n) = ۴T(\sqrt{n}) + \log^۲ n$$

**پاسخ:** در کلاس گفتیم اگر مبنای $\log$ گفته نشده، در مبنای ۲ هست. بنابراین با تغییر متغیر گفته شده و تعریف تابع $S$ داریم.

$$T(۲^m) = ۴T(۲^{m/۲}) + m^۲ \Rightarrow S(m) = ۴S(m/۲) + m^۲$$

بنابر قضیهٔ اصلی داریم

$$S(m) = \Theta(m^۲ \log m) \Rightarrow T(۲^m) = \Theta(m^۲ \log m) = \Theta(\log^۲ n \cdot \log\log n).$$

مشابه این سؤال در تمرین دوم داده شده است.

**نمره‌دهی:** اگر جواب نهایی به صورت $\log^۲ n \log\log n$ یا $m^۲ \log m$ نوشته شود نمره کامل و در غیر این صورت با توجه به راه‌حل نمره داده می‌شود.

پ. **[۵ نمره]** با استفاده از قضیهٔ اصلی، رابطهٔ زیر را حل کنید.

$$T(n) = ۴T(n/۲) + \Theta(n^۲ \log^۲ n)$$

**پاسخ:** با توجه به قضیهٔ اصلی (صورتی که در جلسهٔ سوم کلاس مطرح شد) حالت دوم (تساوی) برقرار است (به طور شهودی می‌توان گفت عبارت‌های لگاریتمی تساوی را به هم نمی‌زنند). بنابر این

$T(n) = \Theta(n^{۲} \log^{۳} n)$ است. حل روابط به قضیهٔ اصلی در تمرین دوم مطرح شده است.

**نمره‌دهی:** اگر جواب نهایی درست باشد نمره کامل وگرنه با توجه به راه‌حل نمره داده خواهد شد: یک نمره برای به‌دست آوردن $L$ و ۴ نمره برای مقایسه و به‌دست آوردن $\Theta$.

**ت. [۵ نمره]** برای مسئله‌ای که طول ورودی آن $n$ و طول خروجی آن $d$ ($۱ \leq d \leq n$) است دو الگوریتم داریم. زمان اجرای الگوریتم اول $\Theta(n+d^{۲})$ و زمان اجرای الگوریتم دوم $\Theta(\delta \cdot n^{۲-\frac{۲}{۷}})$ است که $\delta = \frac{n}{d}$. الگوریتم سومی به این صورت طراحی می‌کنیم که الگوریتم اول و دوم را همزمان اجرا می‌کند و هر کدام زودتر به پایان رسید، جواب آن را برمی‌گرداند. زمان اجرای الگوریتم سوم را در بدترین حالت، فقط برحسب $n$ بیان کنید. (راهنمایی: حالتی را درنظر بگیرید که زمان اجرای دو الگوریتم با هم برابر می‌شود)

**پاسخ:** وقتی $d = ۱$ است الگوریتم اول بهتر است و وقتی $d = n$ باشد الگوریتم دوم بهتر است. همچنین در الگوریتم اول زمان با افزایش $d$ افزایش و در الگوریتم دوم زمان با افزایش $d$ کاهش می‌یابد. بنابراین بدترین حالت وقتی است که زمان اجرای دو الگوریتم با هم برابر شود. در این حالت با صرف‌نظر از $n$ در زمان اجرای الگوریتم اول داریم

$$d^{۲} = n^{۳-(۲/۷)}/d \Rightarrow d^{۳} = n^{۳-(۲/۷)} \Rightarrow d = n^{۱-(۲/۲۱)}.$$

در این حالت زمان اجرای الگوریتم برابر است با $O(n^{۲-۴/۲۱})$ و برای بقیه $d$ها از این مقدار کمتر است. از این رابطه در یکی از لم‌های مقاله زیر استفاده شده است:

Boroujeni, M., Ehsani, S., Ghodsi, M., HajiAghayi, M. and Seddighin, S., 2018. Approximating edit distance in truly subquadratic time: Quantum and MapReduce. In SODA (pp. 1170-1189). SIAM.

**نمره‌دهی:** با توجه به راه‌حل و جواب نهایی نمره داده می‌شود. راه‌حل درست ۴ نمره و جواب درست ۱ نمره دارد.

**سؤال ۳. [۵۰ نمره] پیشنهاد الگوریتم یا داده‌ساختار مناسب.** در هر قسمت از این سؤال از شما خواسته می‌شود یک الگوریتم یا داده‌ساختار مناسب پیشنهاد دهید. این داده‌ساختار یا الگوریتم می‌تواند از مطالب گفته شده در درس، ترکیبی از آن‌ها و یا خلاقیتی باشد. علاوه بر کلیت داده‌ساختار و الگوریتم، زمان اجرا و دلیل انتخاب خود را نیز در ۲ الی ۳ خط (قسمت‌های آ تا ج) و ۴ الی ۵ خط (قسمت‌های چ تا ذ) به طور واضح توضیح دهید. در این سؤال نیاز به اثبات درستی الگوریتم و اثبات زمان اجرای الگوریتم نیست. همچنین در صورتی که زمان اجرای الگوریتم شما زمان سرشکن یا زمان تصادفی است، آن را بیان کنید.

**آ. [۴ نمره]** یک آرایه در ابتدا مرتب بوده، ولی ۱۰ عنصر آن (که نمی‌دانیم کدام عناصر هستند) توسط یک

نفر تغییر داده شده است. چه الگوریتمی برای مرتب‌سازی این آرایه پیشنهاد می‌کنید؟

**پاسخ:** مرتب‌سازی درجی. این مرتب‌سازی برای آرایه‌هایی که تقریباً مرتب هستند مناسب است و این مسئله را در $O(n)$ حل می‌کند.

**نمره‌دهی:** کلیت الگوریتم ۲ نمره و زمان اجرای آن نیز ۲ نمره دارد.

ب. **[۴ نمره]** یک الگوریتم مناسب برای مرتب‌سازی $n$ عدد صحیح که بین ۱ تا $n^{۱۰۰}$ هستند پیشنهاد کنید.

**پاسخ:** مرتب‌سازی مبنایی. همان‌طور که در کلاس گفته شد این مرتب‌سازی این لیست را در $O(n)$ مرتب می‌کند.

**نمره‌دهی:** کلیت اثبات ۳ نمره و زمان اجرا یک نمره دارد. اگر فقط نام الگوریتم برده شده باشد ۱ نمره داده می‌شود. اگر الگوریتم غلط باشد نمره‌ای داده نمی‌شود.

پ. **[۴ نمره]** فرض کنید می‌خواهیم کتاب‌های یک کتابخانه را بر حسب ارتفاعشان مرتب کنیم. ضخامت کتاب‌ها با هم مساوی است و برای هر مقایسه باید دو کتاب را از قفسه‌ها در بیاوریم و با هم مقایسه کنیم. الگوریتم پیشنهادی شما برای مرتب‌کردن کتاب‌های این کتابخانه چیست؟

**پاسخ:** مرتب‌سازی هرمی. زیرا هم زمانش در بدترین حالت $O(\log n)$ است و هم به حافظهٔ اضافه (قفسه‌های خالی) نیاز ندارد. (مرتب‌سازی سریع نیز نمره می‌گیرد)

**نمره‌دهی:** کلیت الگوریتم ۲ نمره، زمان اجرا ۱ نمره و دلیل انتخاب ۱ نمره دارد.

ت. **[۵ نمره]** الگوریتم پیشنهادی شما برای به دست آوردن مسیر بین دو رأس داده شده در یک درخت AVL چیست؟

**پاسخ:** مسیر هر رأس به ریشه را به دست آورده و در دو لیست می‌ریزیم که به خاطر متوازن بودن درخت AVL طول هر کدام $O(\log n)$ است. سپس از انتهای مسیر (ریشه) روی دو لیست حرکت می‌کنیم و قسمت مشترک را حذف می‌کنیم تا به پایین‌ترین جد مشترک برسیم. در انتها با برعکس کردن یک لیست و چسباندن این دو لیست به هم مسیر موردنظر پیدا می‌شود. زمان اجرای این الگوریتم $O(\log n)$ است.

**نمره‌دهی:** مشابه ب.

ث. **[۵ نمره]** برای بررسی این‌که یک درخت داده شده (که رئوس آن نیز رنگ شده‌اند) یک درخت قرمز ـ سیاه هست یا خیر یک الگوریتم پیشنهاد کنید.

**پاسخ:** ابتدا بررسی می‌کنیم که آیا درخت داده شده درخت دودویی جست‌وجو هست یا خیر. این کار را می‌توان با بررسی میان‌ترتیب رئوس انجام داد. سپس خاصیت‌های درخت قرمز سیاه را بررسی می‌کنیم. به غیر از خاصیت سیاه ارتفاع بقیه خاصیت‌ها را در هر رأس می‌توان بررسی کرد و سیاه ارتفاع را نیز با یک بار پیمایش درخت. در مجموع زمان اجرای الگوریتم $O(n)$ می‌باشد.

ج. **[۵ نمره]** یک لیست به طول $n$ از اعداد متمایز داریم که از ابتدا تا یک خانه از آن (که نمی‌دانیم چه خانه‌ای است) صعودی و پس از آن نزولی است. یک الگوریتم کارا برای پیدا کردن مکان عدد داده شدهٔ $x$ در این آرایه (و یا گزارش این‌که $x$ در این لیست وجود ندارد) پیشنهاد کنید.

**پاسخ:** اگر بین هر دو خانه یک فلش فرضی درنظر بگیریم که به سمت خانهٔ بزرگ‌تر اشاره می‌کند، می‌توان با جست‌وجوی دودویی روی این فلش‌ها خانهٔ بیشینه را پیدا کرد. سپس لیست به دو قسمت صعودی و نزولی افراز می‌شود که $x$ را با جست‌وجوی دودویی در هر کدام جست‌وجو می‌کنیم.

چ. **[۵ نمره]** یک واسط صف اولویت دو بعدی به این صورت تعریف می‌شود که کلید هر عنصر یک زوج مرتب $(x, y)$ است. علاوه بر درج (`insert(x, y, item)`) و حذف (`remove(x, y)`) یک عنصر با کلید مشخص، صف اولویت دو بعدی باید بتواند عنصر با کمترین $x$ (`find-min-x()`) و عنصر با کمترین $y$ (`find-min-y()`) را نیز برگرداند. داده‌ساختار پیشنهادی شما برای پیاده‌سازی صف اولویت دوبعدی چیست؟

**پاسخ:** راه‌حل این سؤال یکتا نیست. به طور مثال از دو درخت دودویی جست‌وجوی متوازن (مثل AVL) استفاده می‌کنیم. در درخت اول کلیدها را ابتدا برحسب $x$ و اگر مساوی بودند برحسب $y$ مقایسه می‌کنیم. در درخت دوم کلیدها را ابتدا برحسب $y$ و اگر مساوی بودند برحسب $x$ مقایسه می‌کنیم. درج و حذف در دو درخت به طور عادی قابل انجام است. همچنین برای پیدا کردن عنصر با کمترین $x$ (`find-min-x()`) از درخت اول و برای پیدا کردن عنصر با کمترین $y$ (`find-min-y()`) از درخت دوم استفاده می‌کنیم. تمام عملیات در زمان $O(\log n)$ انجام می‌شود (راه‌حلی بهتری نیز وجود دارد که دو عملیات در $O(1)$ و دو تا در $O(\log n)$ انجام می‌شوند).

**نمره‌دهی:** اگر هر کدام از عملیات $\Theta(n)$ زمان بگیرد، ۱ نمره داده می‌شود. به طور مثال اگر به جای درخت AVL از هرم کمینه یا درخت دودویی جست‌وجوی عادی استفاده شود، `remove` زمان خطی خواهد داشت.

ح. **[۶ نمره]** یک مجموعه $n$تایی از نام‌های کاربری حداکثر ۲۰ حرفی داریم و می‌خواهیم ببینیم چند جفت نام کاربری وجود دارد که دقیقاً در یک حرف با هم تفاوت دارند. الگوریتمی از $O(n)$ طراحی کنید که این کار را انجام دهد. (توجه کنید در این مسئله از درهم‌سازی نمی‌توانید استفاده کنید)

**پاسخ:** به ازای هر کدام از ۲۰ حرف مثل حرف $i$اُم، نام‌های کاربری را به ازای همهٔ حروف به جز حرف $i$اُم با مرتب‌سازی مبنایی در $O(n)$ مرتب می‌کنیم (یعنی این الگوریتم ۲۰ بار مرتب‌سازی دارد). در این صورت رشته‌هایی که فقط در حرف $i$اُم تفاوت دارند پشت سر هم می‌آیند. با پیمایش لیست مرتب‌شده می‌توان این گروه‌های مشابه هم را پیدا کرد. اگر یک گروه با $k$ نام‌کاربری وجود داشته باشد که فقط در حرف $i$ با هم تفاوت دارند، می‌توان $\binom{k}{2}$ را به جواب اضافه کرد. مجموع زمان اجرا $O(n)$ خواهد شد.

راه دیگر (که توسط دانشجویان پیشنهاد شده است): به جای مرتب‌سازی می‌توان نام‌کاربری‌ها را ۲۰ بار دوران داد به صورتی که هر بار حرف $i$اُم حرف آخر قرار بگیرد و آن‌ها را در ترای ریخت و با پیمایش ترای رشته‌های مشابه را پیدا کرد.

**نمره‌دهی:** اگر (چه با استفاده از ترای و چه با استفاده از مرتب‌سازی) فقط اختلاف در حرف آخر را پیدا کنید، ۲ نمره داده می‌شود.

**خ.** **[۶ نمره]** واسط دنبالهٔ پویای محلی به این صورت تعریف می‌شود که علاوه بر لیست، یک مکان‌نما داریم که به یک عضو لیست اشاره می‌کند. عملیات دنبالهٔ پویای محلی به صورت زیر است:

- `at(i)`: برگرداندن عنصر $i$اُم (برای $i$ دلخواه)
- `cur-index()`: برگرداندن اندیس مکان‌نما.
- `move-left()` و `move-right()`: حرکت مکان‌نما یک واحد به چپ یا یک واحد به راست.
- `delete-here()`: حذف عنصری که مکان‌نما روی آن است (و جابه‌جایی عناصر بعدی به چپ تا جای خالی پر شود).
- `insert-here(x)` درج یک عنصر جدید در سمت چپ عنصری که مکان‌نما روی آن است (و جابه‌جا کردن عناصر بعدی) و حرکت مکان‌نما به روی عنصر جدید.

چه داده‌ساختاری برای این واسط پیشنهاد می‌کنید؟

**پاسخ:** دو پشته یکی برای قسمتی از لیست که سمت راست مکان‌نما است و یکی برای قسمتی از لیست که سمت چپ مکان‌نما است می‌گیریم (به صورتی که عنصر مجاور مکان‌نما در بالای پشته قرار گیرد). همچنین عنصر جاری را در یک متغیر جداگانه ذخیره می‌کنیم. پشته‌ها را نیز با آرایهٔ پویا پیاده‌سازی می‌کنیم برگرداندن عنصر جاری بدیهی است. اندیس مکان‌نما برابر با تعداد عناصر پشته چپ به اضافهٔ یک است. حرکت به چپ و راست با یک pop از یک پشته و یک push به پشته دیگر قابل انجام است. همچنین حذف با یک pop و درج با یک push قابل انجام است. بنابراین تمام عملیات خواسته شده در $O(۱)$ قابل انجام است.

**د.** **[۶ نمره]** در یک مهمانی $n$ مرد با بارانی‌های مشابه که فقط اندازه‌های متفاوت دارند وارد می‌شوند و آن‌ها را در یک اتاق قرار می‌دهند. در پایان مهمانی مشخص نیست کدام بارانی برای چه کسی هست. می‌خواهیم بارانی هر فرد را مشخص کنیم. تنها کاری که مجاز به انجام آن هستیم این است که یک بارانی را بر تن یک نفر امتحان کنیم که یا اندازه است، یا کوچک است و یا بزرگ. فرض کنید اندازهٔ بارانی‌ها و افراد با هم متفاوت است و نمی‌توانیم دو بارانی و یا دو نفر را با هم مقایسه کنیم. چه الگوریتمی برای حل این مسئله پیشنهاد می‌کنید؟

**پاسخ:** از الگوریتمی مشابه مرتب‌سازی سریع تصادفی استفاده می‌کنیم. یکی از افراد را به تصادف به عنوان لولا (محور) درنظر می‌گیریم و تمام بارانی‌ها را با او امتحان می‌کنیم. بنابراین یک بارانی اندازه، تعدادی کوچکتر و تعدادی بزرگتر می‌شود. سپس با استفاده از بارانی اندازه و امتحان کردن آن با همهٔ افراد، افراد نیز به همین سه دسته افراز می‌شوند. فرد انتخابی و بارانی آن کنار گذاشته شده و دو دستهٔ بارانی‌ها و افراد کوچکتر و بارانی‌ها و افراد بزرگتر را به صورت بازگشتی می‌توانیم حل کنیم. زمان اجرا به صورت تصادفی (میانگین) $O(n \log n)$ خواهد شد.

**سؤال ۴.** **[۱۷ نمره]** یک مجموعه از اعداد را $k$ـ محسوس می‌گوییم اگر جمع هر زیرمجموعهٔ $k$ عضوی از آن بزرگ‌تر یا مساوی هر کدام از اعضای مجموعه باشد. فرض کنید $M$ یک مجموعه از اعداد متمایز است و می‌خواهیم بزرگترین زیرمجموعهٔ $k$ـ محسوس آن را پیدا کنیم. فرض کنید بزرگ‌ترین زیرمجموعهٔ $k$ـ محسوس از $M$ یکتا است. در پاسخ به موارد زیر اثبات دقیق ادعای خود را بیان کنید.

آ. **[۷ نمره]** ثابت کنید بزرگ‌ترین زیرمجموعهٔ $k$ـ محسوس از $M$ محدب است، یعنی اگر شامل $a$ و $b$ باشد، شامل هر عضو $c$ که $a < c < b$ و $c \in M$ هم باشد.

**پاسخ:** فرض کنید $A$ بزرگ‌ترین زیرمجموعهٔ $k$ـ محسوس از $M$ و $s$ مجموع $k$ عضو کوچکتر $A$ باشد. طبق فرض $k$ـ محسوس بودن $A$، $s$ بزرگ‌تر یا مساوی هر کدام از عناصر $A$ است. همچنین بنابر تعریف $s$، مجموع هر $k$ عضو $A$ حداقل $s$ است. بنابراین شرط لازم و کافی $k$ـ محسوس بودن $A$ این است که اعضای آن کوچکتر از $s$ نباشند. از طرفی اعضای $s$ در $M$ متوالی هستند، در غیر این صورت می‌توان کوچکترین عنصر $A$ را حذف و یکی از عناصر جا افتاده را اضافه کرد و $A$ جدید همچنان $k$ـ محسوس است. این موضوع با یکتا بودن $A$ در تناقض است.

حال فرض خلف کنید که $A$ محدب نیست، بنابراین سه عنصر $a < b < c$ از $M$ را درنظر بگیرید که $a, c \in A$ و $b \notin A$. از طرفی $s \leq a < b$ پس $b$ را می‌توان به $A$ اضافه کرد و شرط $k$ـ محسوس بودن باقی می‌ماند (طبق متوالی بودن اعضای $s$، اضافه کردن عنصر $b$ روی $s$ تأثیر نمی‌گذارد). بنابراین، $A$ بزرگ‌ترین زیرمجموعهٔ $k$ـ محسوس از $M$ نیست و فرض خلف غلط است. پس $A$ محدب است.

ب. **[۱۰ نمره]** الگوریتمی با زمان اجرای $O(n \log n)$ طراحی کنید که بزرگ‌ترین زیرمجموعهٔ $k$ـ محسوس از $M$ را پیدا کند. درستی و زمان اجرای الگوریتم خود را ثابت کنید.

**پاسخ:** با توجه به محدب بودن بزرگ‌ترین زیرمجموعهٔ $k$ محسوس به صورت زیر عمل می‌کنیم. ابتدا $M$ را مرتب می‌کنیم. سپس تمام عناصر $M$ را به عنوان کوچکترین عنصر مجموعه امتحان می‌کنیم و بزرگ‌ترین عنصر را با جست‌وجوی دودویی به دست می‌آوریم. برای هر عنصر انتخاب شده به عنوان کوچکترین عنصر، $s$ را که حاصل‌جمع $k$ عنصر کوچکتر است را به دست می‌آوریم. اگر به ترتیب جلو

برویم، $s$ را از مرحلهٔ قبل با حذف یک عنصر و اضافه کردن یک عنصر در $O(۱)$ به دست می‌آوریم. سپس به دنبال بزرگ‌ترین عنصری از $M$ می‌گردیم که کمتر مساوی $s$ باشد. تمام عناصر بین کوچکترین و بزرگترین عنصر انتخاب شده یک مجموعه $k$ـ محسوس است و بزرگترین این مجموعه‌ها، بزرگ‌ترین مجموعهٔ $k$ـ محسوس $M$ خواهد بود.

زمان اجرای الگوریتم برای مرتب‌سازی $O(n \log n)$ و برای قسمت بعدی نیز $n \cdot (O(۱) + O(\log n)) =$ $O(n \log n)$ است. بنابراین زمان اجرای کل الگوریتم $O(n \log n)$ است.

نسخه سخت‌تری از سؤال که قرار بود مطرح شود به این صورت بود که اعداد $M$ دارای محدودیت هستند (اعداد صحیح بین ۱ تا $n^{۱۰۰}$) و بنابراین زمان اجرایی مرتب‌سازی $O(n)$ می‌شود. همچنین با کمی تلاش می‌توان جست‌وجوی دودویی را حذف کرد و مکان‌نمایی برای بزرگ‌ترین عنصر $k$ـ محسوس درنظر گرفت و نشان داد این مکان‌نما در هر مرحله به عقب برنمی‌گردد. بنابراین قسمت دوم را نیز می‌توان در زمان $O(n)$ حل کرد.

موفق باشید